KANN AUDITS

SECURITY REVIEW

FOR APTREE



PREPARED FOR
APTree

PREPARED BY
Kann Audits

DATE REPORTED
13 April 2026

INSIDE THIS REPORT

CONTENTS

01	AUDIT SUMMARY	03
02	PROTOCOL OVERVIEW	04
03	SCOPE	05
04	AUDITORS	06
05	METHODOLOGY	07
06	RISK CLASSIFICATION	08
07	FINDINGS SUMMARY	09
08	DETAILED FINDINGS	10
	H-01 Locking module never takes custody of AET, so the lock period, early-withdrawal cap and emergency forfeiture can be bypassed	10
	M-01 Only one claimant per MoneyFi processing cycle can withdraw, and anyone can block withdrawals with `withdraw(0)`	12
	M-02 Locking exits and `request_and_withdraw` wrappers expect MoneyFi to settle in the same transaction and revert	14
	M-03 `locking::emergency_unlock` burns only the AET needed for the payout, so the forfeited yield stays redeemable by the user	15
	M-04 GuaranteedYieldLocking emergency unlock strands the forfeited-yield AET in the controller, untracked and unredeemable	16
	L-01 Emergency-unlock events report `yield_forfeited = 0` and overstate `aet_burned`	17
	I-01 `CASHBACK_LOW_THRESHOLD` is 100 times the documented value	18
	I-02 Unused fields, error constants and helpers	19
	I-03 The locking admin cannot be transferred	20
	I-04 Admin state changes emit no events	21
	I-05 The bridge's `provider` parameter is accepted but ignored	22
	I-06 The slippage check on locking exits can never fail	23
09	DISCLAIMER	24
10	ABOUT KANN AUDITS	25

01 - OVERVIEW

AUDIT SUMMARY

APTree engaged Kann Audits to perform a security review of the APTree Earn smart contracts. From 6 – 12 April 2026, a team of 19 security researchers performed a manual review of the in-scope contracts and additionally triaged findings generated by the Kann AI Security Model. All findings identified during the assessment are documented in this report.

TOTAL FINDINGS

12 ISSUES

CRITICAL	–
HIGH	01
MEDIUM	04
LOW	01
INFORMATIONAL	06

ENGAGEMENT DETAILS

PREPARED FOR	APTree
PREPARED BY	Kann Audits
DATE REPORTED	13 April 2026
REVIEW PERIOD	6 – 12 April 2026
LANGUAGE	Move
CHAIN	Aptos
SCOPE	2,180 nSLOC
TIMELINE	7 days
METHODS	Manual review, Kann AI Security Model

02 - WHAT THE PROTOCOL DOES

PROTOCOL OVERVIEW

APTree Earn is a yield access layer built on Aptos that simplifies DeFi participation by abstracting complexity away from the user.

Users deposit USDt through a bridge that forwards the funds to MoneyFi's yield strategies and mints AET share tokens, which track the vault's share price. Withdrawals take two steps: a request burns the user's AET, mints AEWT withdrawal tokens and queues the withdrawal with MoneyFi; once MoneyFi has processed it, a second call burns the AEWT and pays out the USDt.

On top of the bridge, the reviewed scope includes a time-locked deposit module with 3, 6 and 12-month tiers, a limited early-withdrawal allowance and an emergency unlock that forfeits accrued yield; GuaranteedYieldLocking, which pays a fixed yield upfront as cashback from a prefunded vault and holds the AET until maturity; and the Glade entry points, which combine these deposits and withdrawals with Panora token swaps.

03 - WHAT WE REVIEWED

SCOPE

REPOSITORY	moneyfi-aptree-earn-contracts
AUDITED COMMIT	1e816001d038f1a3b435abea3c4e8f9406afec89

#	FILE	NSLOC
01	contracts/bridge/sources/bridge.move	40
02	contracts/bridge/sources/moneyfi_adapter.move	294
03	contracts/locking/sources/locking.move	501
04	contracts/guaranteed-yield/sources/GuaranteedYieldLocking.move	869
05	contracts/glade/sources/glade_flexible.move	156
06	contracts/glade/sources/glade_guaranteed.move	157
07	contracts/glade/sources/swap_helpers.move	163
	Total	**2,180**

04 - THE TEAM ON THIS ENGAGEMENT

AUDITORS

CO	count-sum SECURITY RESEARCHER	RA	Razkky SECURITY RESEARCHER
ST	stranger8732 SECURITY RESEARCHER	HI	Himanshu772005 SECURITY RESEARCHER
LU	lunarius SECURITY RESEARCHER	JA	jayjoshix SECURITY RESEARCHER
0X	0xhalas SECURITY RESEARCHER	VO	Volodymyr SECURITY RESEARCHER
AT	Atiq Ishrak SECURITY RESEARCHER	GR	GraySon SECURITY RESEARCHER
RA	Rare_one SECURITY RESEARCHER	SH	Shreyash SECURITY RESEARCHER
IB	Ibukun SECURITY RESEARCHER	JU	JustUzair SECURITY RESEARCHER
PH	PhlavorPhlay SECURITY RESEARCHER	SH	Shieldrey SECURITY RESEARCHER
DE	DeveloperX SECURITY RESEARCHER	AL	AlexScherbatyuk SECURITY RESEARCHER
YU	yusufthebdev SECURITY RESEARCHER		

05 - HOW WE WORK

METHODOLOGY

An engagement with Kann Audits involves a team of security researchers performing independent reviews of the codebase at the start of the engagement, followed by collaborative review sessions where auditors share findings and explore additional attack vectors to improve overall coverage.

The review included a line-by-line manual inspection of the in-scope codebase, along with analysis of the protocol architecture, access control mechanisms and state transitions. The team also assessed the code against common smart contract security best practices and industry standards.

STEP BY STEP

OUR SECURITY PROCESS

01 **Kickoff & Context**
A kickoff call to understand architecture, trust boundaries, threat assumptions and intended behaviour.

02 **Scope & Preparation**
Confirm the commit, files in scope, dependencies and the riskiest areas. We open a private channel between your team and our auditors.

03 **AI First Scan**
Kann AI, our proprietary model built to find security bugs, scans the codebase before the manual review starts.

04 **Expert Security Review**
Line-by-line analysis of exploit paths, invariant breaks, edge cases, accounting and integration risks.

05 **Findings & Proofs**
Each issue is confirmed, rated and backed by a proof of concept or a precise reproduction path.

06 **Mitigation Review**
We review every fix, confirm it resolves the root cause and flag partial or incorrect mitigations.

COVERAGE

THE REVIEW SPECIFICALLY CONSIDERED

- Testing against common and uncommon attack vectors
- Contract logic matching the client's intended behaviour
- Manual line-by-line review by experienced auditors
- Architecture and design analysis for secure component interactions and clear trust boundaries
- Compliance with industry best practices and known smart contract security standards
- Cross-referencing patterns with industry-standard implementations
- Access control verification
- State transitions and logic, to detect inconsistencies, unexpected behaviour or exploitation paths

06 - HOW WE RATE ISSUES

RISK CLASSIFICATION

Severity combines the **impact** of an issue if exploited with the **likelihood** that the conditions for exploitation are met.

CRITICAL	Directly exploitable with few or no preconditions, leading to theft or permanent loss of funds or loss of control of the system. Must be fixed before deployment.
HIGH	Directly exploitable vulnerabilities that can lead to significant material loss or critical impact on the system and require immediate fixing.
MEDIUM	Moderate loss of funds or impact on multiple users, but only under specific conditions. Exploitable without privileged access, but typically requires certain assumptions or edge cases.
LOW	Issues with limited impact that do not put funds directly at risk, such as edge cases or deviations from documented behaviour.
INFORMATIONAL	Non-exploitable findings such as best-practice deviations, cosmetic issues or documentation and compliance notes. They do not need urgent remediation but improve code quality and maintainability.

07 - ALL ISSUES

FINDINGS SUMMARY

CRITICAL	HIGH	MEDIUM	LOW	INFO
-	01	04	01	06

TOTAL 12 FIXED 06 ACKNOWLEDGED 06

ID	TITLE	SEVERITY	STATUS
H-01	Locking module never takes custody of AET, so the lock period, early-withdrawal cap and emergency forfeiture can be bypassed	HIGH	FIXED
M-01	Only one claimant per MoneyFi processing cycle can withdraw, and anyone can block withdrawals with `withdraw(0)`	MEDIUM	FIXED
M-02	Locking exits and `request_and_withdraw` wrappers expect MoneyFi to settle in the same transaction and revert	MEDIUM	FIXED
M-03	`locking::emergency_unlock` burns only the AET needed for the payout, so the forfeited yield stays redeemable by the user	MEDIUM	FIXED
M-04	GuaranteedYieldLocking emergency unlock strands the forfeited-yield AET in the controller, untracked and unredeemable	MEDIUM	FIXED
L-01	Emergency-unlock events report `yield_forfeited = 0` and overstate `aet_burned`	LOW	FIXED
I-01	`CASHBACK_LOW_THRESHOLD` is 100 times the documented value	INFORMATIONAL	ACKNOWLEDGED
I-02	Unused fields, error constants and helpers	INFORMATIONAL	ACKNOWLEDGED
I-03	The locking admin cannot be transferred	INFORMATIONAL	ACKNOWLEDGED
I-04	Admin state changes emit no events	INFORMATIONAL	ACKNOWLEDGED
I-05	The bridge's `provider` parameter is accepted but ignored	INFORMATIONAL	ACKNOWLEDGED
I-06	The slippage check on locking exits can never fail	INFORMATIONAL	ACKNOWLEDGED

08 - ISSUE BY ISSUE

DETAILED FINDINGS

H-01 Locking module never takes custody of AET, so the lock period, early-withdrawal cap and emergency forfeiture can be bypassed

SEVERITY ● HIGH STATUS ● FIXED

LOCATION `locking.move#L225` `locking.move#L316` `moneyfi_adapter.move#L194-L255`

DESCRIPTION

The locking product promises that deposits stay locked for 90, 180 or 365 days, that only a 2-5% early-withdrawal allowance is available before maturity, and that an emergency unlock forfeits accrued yield (RFC-002 §1.2, §6, §8). None of this is enforced on-chain.

`deposit_locked` and `add_to_position` deposit through the bridge with the user's own signer, so the AET share tokens are minted into the user's wallet. The locking module only records the amount in `LockPosition.aet_amount`: no AET is moved to the locking controller, and `LockConfig.signer_cap` is never used.

LOCKING.MOVE

```
224  // Perform the deposit through MoneyFiBridge
225  MoneyFiBridge::deposit(user, amount);
```

The bridge's `request` and `withdraw` entry points are public and only check the caller's own AET balance (`moneyfi_adapter.move:216-218`). They have no reference to the locking module, and AET is a freely transferable fungible asset, so any locker can redeem their AET directly at any time. `GuaranteedYieldLocking` is not affected: it deposits with its controller signer and holds the AET itself.

IMPACT

- The lock duration, the early-withdrawal cap and the emergency forfeiture cannot be enforced for any locking position, which defeats the product's stated goal of predictable liquidity (RFC-002 §1.2).
- Yield that RFC-002 §8.1 assigns to the remaining holders on early exit can be kept by the exiting user.
- The abandoned `LockPosition` stays on record: `get_user_total_locked_value` and `get_early_withdrawal_available` keep reporting value for it, and later `withdraw_unlocked` or `emergency_unlock` calls abort with `ELP_AMOUNT_INSUFFICIENT`.

No other user's principal is taken, since locking pays no extra yield (RFC-002 §2). The issue is rated High because the lock is the module's only security guarantee and it is void for every position.

RECOMMENDATION

- Escrow AET in the locking controller: after `MoneyFiBridge::deposit`, move the minted AET into the controller (measuring the balance delta), or add a `friend`-only adapter function that mints directly to the controller.
- Track AET per position in the controller and run every exit with the controller signer (`signer_cap`) through a two-step request/complete flow (see M-02), paying the user on completion.
- Optionally, restrict the redeem paths used by locking to `friend` modules.

REMEDIATION ● FIXED

M-01 Only one claimant per MoneyFi processing cycle can withdraw, and anyone can block withdrawals with `withdraw(0)`

SEVERITY MEDIUM STATUS FIXED LOCATION moneyfi_adapter.move#L194-L255 moneyfi_adapter.move#L257-L294

DESCRIPTION

Every user's withdrawal request is filed with MoneyFi under one account, the bridge's reserve. MoneyFi stores withdrawal requests per wallet account, so it sees a single aggregated withdrawal state for the whole protocol.

`withdraw_fungible` burns the caller's AEWT, always calls `vault::withdraw_requested_amount` on the reserve, and only then pays the caller a fixed `amount` from the reserve:

MONEYFI_ADAPTER.MOVE

```
270  // burn withdrawal tokens
271  let asset = get_withdrawal_metadata(reserve_address);
272  let from_wallet = primary_fungible_store::primary_store(address_of(user), asset);
273  fungible_asset::burn_from(&withdrawal_state.burn_ref, from_wallet, amount);
274
275  // this will withdraw all pending requesteed amounts that's available
276  vault::withdraw_requested_amount(&reserve_signer, token);
277
278  primary_fungible_store::transfer<Metadata>(
279      &reserve_signer,
280      token,
281      address_of(user),
282      amount
283  );
```

According to MoneyFi's integration documentation, `withdraw_requested_amount` withdraws the entire available amount and requires it to be greater than zero. Nothing links the caller's AEWT to their own processed request, so the first caller after each processing event sweeps everything into the reserve. Every other claimant's call then aborts at line 276, even though the reserve already holds their USDt, until MoneyFi makes a new amount available.

Zero-amount burns and transfers do not abort, so `withdraw(0)` burns nothing, still sweeps the available amount and pays nothing. There is no per-user claim ledger, and the `ECLAIMS_*` error codes declared in the module are never used.

IMPACT

- Withdrawals stall protocol-wide: bridge AEWT redemptions, the second step of GuaranteedYieldLocking exits (`withdraw_guaranteed`, `withdraw_emergency_guaranteed`) and the locking exits. In normal operation at most one claimant per processing cycle succeeds.
- Any account with an AEWT primary store, which can hold a zero balance, can keep everyone else's withdrawals blocked by calling `withdraw(0)` after each processing event, paying only gas.
- A claimant whose own request has not been processed yet can be paid from funds processed for others.

No funds are lost: the reserve balance plus MoneyFi's pending amount still back the AEWT supply, and blocked users can recover by winning a later race. The impact grows if MoneyFi processes requests rarely or in large batches.

RECOMMENDATION

- In `withdraw_fungible`, read the reserve's withdrawal state (`wallet_account::get_withdrawal_state`) and call `withdraw_requested_amount` only when the available amount is greater than zero. Pay from the reserve whenever its balance covers `amount`, and abort with a clear error otherwise.
- Add a per-user claim ledger (FIFO by request ID) so processed funds are paid to the requesters they belong to.
- Reject `amount == 0`.
- Optionally, add a permissionless `sync()` that pulls available funds into the reserve without paying anyone.

REMEDIATION ● FIXED

M-02 Locking exits and `request_and_withdraw` wrappers expect MoneyFi to settle in the same transaction and revert

SEVERITY ● MEDIUM STATUS ● FIXED

LOCATION `locking.move#L387-L388` `locking.move#L439-L440` `locking.move#L498-L499` `bridge.move#L45-L52` `glade_flexible.move#L142`

DESCRIPTION

`locking::withdraw_early`, `withdraw_unlocked` and `emergency_unlock`, as well as `bridge::request_and_withdraw` and `glade_flexible::request_and_withdraw`, call the adapter's `request` and then `withdraw` in the same transaction:

BRIDGE.MOVE

```
45  public entry fun request_and_withdraw(
46      user: &signer,
47      amount: u64,
48      min_share_price: u128
49  ) {
50      moneyfi_adapter::request(user, amount, min_share_price);
51      moneyfi_adapter::withdraw(user, amount)
52  }
```

MoneyFi withdrawals are asynchronous: a request is queued for an off-chain backend, and `withdraw_requested_amount` succeeds only once an amount has been made available. A request created in the current transaction can never be available within that transaction. The protocol's own GuaranteedYieldLocking module handles this correctly with a request step and a separate, later withdrawal step.

IMPACT

- All three locking exits and both convenience wrappers fail in normal operation.
- If another user's processed funds happen to be available, the call succeeds by paying this user out of those funds, which is the queue jump described in M-01.
- No funds are lost because the revert is atomic. Users can still exit through `bridge::request` followed later by `bridge::withdraw`, but only because locking never takes custody of their AET (H-01). If H-01 were fixed with escrow but without a two-step exit, locked funds would become unreachable.

`glade_flexible::withdraw`, which performs only the second step, is not affected.

RECOMMENDATION

- Give `locking` a two-step exit modelled on GuaranteedYieldLocking: `request_withdraw_locked(position_id)` records a pending exit, and `complete_withdraw_locked(position_id)` performs the withdrawal after MoneyFi has processed it, using controller custody (H-01).
- Remove `bridge::request_and_withdraw` and `glade_flexible::request_and_withdraw`, or restrict them to providers that settle synchronously.

REMEDIATION ● FIXED

M-03 `locking::emergency_unlock` burns only the AET needed for the payout, so the forfeited yield stays redeemable by the user

SEVERITY MEDIUM STATUS FIXED LOCATION `locking.move#L476-L499`

DESCRIPTION

An emergency unlock is meant to return at most the principal and forfeit any yield above it to the remaining holders. RFC-002 §8.1 says the forfeited yield "stays in pool (benefits remaining users)", §8.4 recommends burning the excess AET, and the locking test plan expects "Excess AET is burned" (TEST-CASES-LOCKING §5.3).

The implementation pays `min(principal, current_value)` (line 482) and requests only that amount from the bridge, which burns `floor(payout × 1e9 / share_price)` AET. The position is then deleted, but the rest of its AET is never burned or confiscated and remains in the user's wallet. The comment on line 497 is incorrect.

LOCKING.MOVE

```
493  // Remove position
494  vector::swap_remove(&mut user_positions.positions, index);
495
496  // Perform withdrawal for payout amount only
497  // The forfeited yield stays in the pool (AET is burned, value remains)
498  MoneyFiBridge::request(user, payout, share_price);
499  MoneyFiBridge::withdraw(user, payout);
```

IMPACT

The emergency-exit penalty is not applied: the user keeps their accrued yield, and the remaining holders lose the windfall the design promises. The loss per exit is bounded by that position's accrued yield, and no principal is at risk. The same value can currently also be extracted through H-01, but this defect survives fixes to H-01 and M-02: with escrow, the residual AET would instead be orphaned in the controller, as in M-04.

RECOMMENDATION

- Burn the excess AET (`aet_amount` minus the AET burned for the payout) as part of the emergency exit. Once custody is fixed (H-01), do this from the controller store through a `friend`-only adapter burn function.
- Alternatively, request the full `current_value` and route the excess to the pool or treasury as specified.
- Emit the AET actually burned (see L-01).

REMEDIATION FIXED

M-04 GuaranteedYieldLocking emergency unlock strands the forfeited-yield AET in the controller, untracked and unredeemable

SEVERITY MEDIUM STATUS FIXED

LOCATION `GuaranteedYieldLocking.move#L717-L760` `GuaranteedYieldLocking.move#L763-L857`

DESCRIPTION

GuaranteedYieldLocking holds every position's AET in its controller. On an emergency unlock it requests only `base_payout = min(principal, current_value)` from MoneyFi, which burns the AET for that amount. The AET representing the yield above principal stays in the controller, but `total_aet_held` is reduced by the whole position's AET:

GUARANTEEDYIELDLOCKING.MOVE

```
754  // Update global stats
755  config.total_locked_principal -= principal;
756  config.total_aet_held -= position.aet_amount;
```

No function in the module ever requests or burns that untracked balance, because every later request is sized to a single position. This contradicts the module's own documentation: the comment on `withdraw_emergency_guaranteed` says the treasury receives "the remainder (cashback recovery + any forfeited yield)", and RFC-003 §4.3 says the "protocol keeps all yield above principal". In practice the treasury receives only the cashback clawback.

IMPACT

- The forfeited yield of every emergency unlock with `current_value > principal`, which is expected protocol revenue, is stranded. The amount grows with yield and usage.
- Because the AET is never burned, it does not raise the share price for the other holders either.
- `get_protocol_stats` under-reports the AET the controller actually holds.

Users lose nothing. The value can only be recovered through a package upgrade that adds a sweep.

RECOMMENDATION

- Request the full `current_value` in the first step and send `current_value - payout` to the treasury in the second step.
- Alternatively, track the surplus explicitly and add an admin `sweep` that redeems the untracked controller AET (the controller's AET balance minus `total_aet_held`) to the treasury.
- In both cases, decrement `total_aet_held` by the AET actually burned.

REMEDIATION FIXED

L-01 Emergency-unlock events report `yield_forfeited = 0` and overstate `aet_burned`

SEVERITY ● LOW STATUS ● FIXED LOCATION `GuaranteedYieldLocking.move#L812-L856` `locking.move#L507`

DESCRIPTION

`withdraw_emergency_guaranteed` reconstructs the position's value at request time from `pending.withdrawal_amount`. That field already holds the capped `base_payout` (at most the principal), so the reconstruction always returns the capped value and `yield_forfeited` is always 0, including exactly when yield was forfeited.

GUARANTEEDYIELDLOCKING.MOVE

```
812  // Reconstruct event data from pending state
813  let principal = pending.position.principal;
814  let current_value = pending.withdrawal_amount + (
815      if (pending.position.aet_amount > 0) {
816          // If base_payout < current_value, yield was forfeited (stayed in pool)
817          // We stored base_payout as withdrawal_amount, so compute original current_value
818          let share_price_at_request = ((pending.withdrawal_amount as u128) * AET_SCALE) /
     (pending.position.aet_amount as u128);
819          let original_current_value = ((pending.position.aet_amount as u128) *
     share_price_at_request / AET_SCALE) as u64;
820          if (original_current_value > principal) { original_current_value - principal } else { 0 }
821      } else { 0 }
822  );
823
824  let yield_forfeited =
825      if (current_value > principal) {
826          current_value - principal
827      } else { 0 };
```

Both modules also report the whole position's AET as `aet_burned` (`GuaranteedYieldLocking.move:852`, `locking.move:507`), although only the AET equivalent to the payout is burned (see M-03 and M-04). The `yield_forfeited` value emitted by `locking` is computed correctly.

IMPACT

Indexers, dashboards and treasury revenue analytics built on these events under-report forfeited yield and over-report AET burned. There is no on-chain effect on funds. For example, for a position with a 1,000 USDt principal unlocked at a share price of 1.1, `GuaranteedEmergencyUnlock` emits `yield_forfeited = 0` (true value about 100 USDt) and `aet_burned` of 1,000 AET (true value 909.090909 AET).

RECOMMENDATION

- Store the request-time `current_value` (or the forfeited yield) and the AET actually burned in `PendingUnlock`, and emit those values.
- In `locking::emergency_unlock`, emit `floor(payout × 1e9 / share_price)` as `aet_burned`.

REMEDIATION ● FIXED

I-01 `CASHBACK_LOW_THRESHOLD` is 100 times the documented value

SEVERITY ● INFORMATIONAL STATUS ● ACKNOWLEDGED

LOCATION `GuaranteedYieldLocking.move#L47` `GuaranteedYieldLocking.move#L470-L480`

DESCRIPTION

`CASHBACK_LOW_THRESHOLD` is commented as 100 USDT but equals 10,000 USDt at 6 decimals, so the `CashbackVaultLow` warning fires at 100 times the documented level.

GUARANTEEDYIELDLOCKING.MOVE

```
46  // Warning threshold for cashback vault (configurable)
47  const CASHBACK_LOW_THRESHOLD: u64 = 100_00000000; // 100 USDT
```

RECOMMENDATION

Set the constant to `100_000_000` (100 USDt at 6 decimals), or correct the comment if 10,000 USDt is the intended threshold.

REMEDIATION ● ACKNOWLEDGED

I-02 Unused fields, error constants and helpers

SEVERITY ● INFORMATIONAL STATUS ● ACKNOWLEDGED

LOCATION `bridge.move#L8-L21` `locking.move#L44` `locking.move#L87` `moneyfi_adapter.move#L30-L36` `swap_helpers.move#L141-L181`

DESCRIPTION

`bridge::State.signer_cap` and `LockConfig.signer_cap` are stored but never used. Several error constants are never referenced, including `ECLAIMS_DO_NOT_EXIST`, `ECLAIMS_ARE_LESS`, `ELP_AMOUNT_DOES_NOT_EXIST`, `ENOT_POSITION_OWNER`, `EUNSUPPORTED_PROVIDER` and `EOPERATION_NOT_PERMITTED`. The helpers `check_and_deposit_fa_to_address_opt` and `check_and_deposit_coin_to_address_opt` are never called.

RECOMMENDATION

Remove the unused fields, constants and helpers, or use them where they were intended, for example `LockConfig.signer_cap` for controller custody (H-01).

REMEDIATION ● ACKNOWLEDGED

I-03 The locking admin cannot be transferred

SEVERITY INFORMATIONAL STATUS ACKNOWLEDGED LOCATION `locking.move#L186-L192`

DESCRIPTION

`LockConfig.admin` is fixed at initialisation and there is no function to change it, while GuaranteedYieldLocking supports a propose/accept admin transfer.

RECOMMENDATION

Add a two-step (propose/accept) admin transfer, as in GuaranteedYieldLocking.

REMEDIATION ACKNOWLEDGED

I-04 Admin state changes emit no events

SEVERITY ● INFORMATIONAL STATUS ● ACKNOWLEDGED

LOCATION `locking.move#L518-L539` `GuaranteedYieldLocking.move#L919-L936`

DESCRIPTION

`locking::set_tier_limit`, `locking::set_locks_enabled` and `GuaranteedYieldLocking::admin_withdraw_cashback_vault` change state silently, which makes off-chain monitoring harder.

RECOMMENDATION

Emit an event for each of these, with the previous and new value or the amount withdrawn.

REMEDIATION ● ACKNOWLEDGED

I-05 The bridge's `provider` parameter is accepted but ignored

SEVERITY ● INFORMATIONAL STATUS ● ACKNOWLEDGED LOCATION `bridge.move#L23-L43`

DESCRIPTION

`bridge::deposit` and `bridge::withdraw` take a `provider` argument that is never read; every call is routed to MoneyFi.

RECOMMENDATION

Validate `provider` and abort with `EUNSUPPORTED_PROVIDER` for unknown values, or remove the parameter until other providers are supported.

REMEDIATION ● ACKNOWLEDGED

I-06 The slippage check on locking exits can never fail

SEVERITY ● INFORMATIONAL STATUS ● ACKNOWLEDGED

LOCATION `locking.move#L387` `locking.move#L439` `locking.move#L498` `moneyfi_adapter.move#L204`

DESCRIPTION

Locking passes the share price it has just read as `min_share_price`, so the adapter's check `share_price ≥ min_share_price` always passes and users cannot set their own minimum.

LOCKING.MOVE

```
385  // Perform withdrawal through MoneyFiBridge
386  // First request, then withdraw
387  MoneyFiBridge::request(user, amount, share_price);
388  MoneyFiBridge::withdraw(user, amount);
```

RECOMMENDATION

Let users pass their own `min_share_price` to the locking exit functions and forward it to the adapter.

REMEDIATION ● ACKNOWLEDGED

09 - PLEASE READ

DISCLAIMER

A security audit can never guarantee the complete absence of vulnerabilities. Audits are a time, resource and expertise-bound effort in which we aim to identify as many issues as possible.

This review covers only the files listed in the Scope section at the audited commit. Code changed after that commit, external dependencies, deployment configuration and operational practices may introduce additional risk.

10 - WHO WE ARE

ABOUT KANN AUDITS

Kann Audits is a Web3 security audit company helping DeFi protocols, blockchain networks, and infrastructure teams build securely at every stage, from development to launch and beyond.

EXPERT SECURITY AUDITS

Human-led smart contract security audits built around the architecture, business logic, integrations and risk profile of complex Web3 protocols.

AI SECURITY SCAN

Kann AI extracts Solidity call graphs and returns function-level security signals for expert validation.

FORMAL VERIFICATION

Define security properties for critical protocol behaviour and evaluate whether the implementation satisfies those specifications.

INCIDENT RESPONSE

A direct triage path for suspected exploits, critical vulnerabilities, suspicious transactions and urgent remediation review.

KANN AUDITS

WEB3 SECURITY AUDITS

GET IN TOUCH

WEBSITE	kannaudits.com
PORTFOLIO	github.com/KannAudits
EMAIL	contact@kannaudits.com
X	@KannAudits
TELEGRAM	@KannAudits

© 2026 Kann Audits. Prepared for APTree.